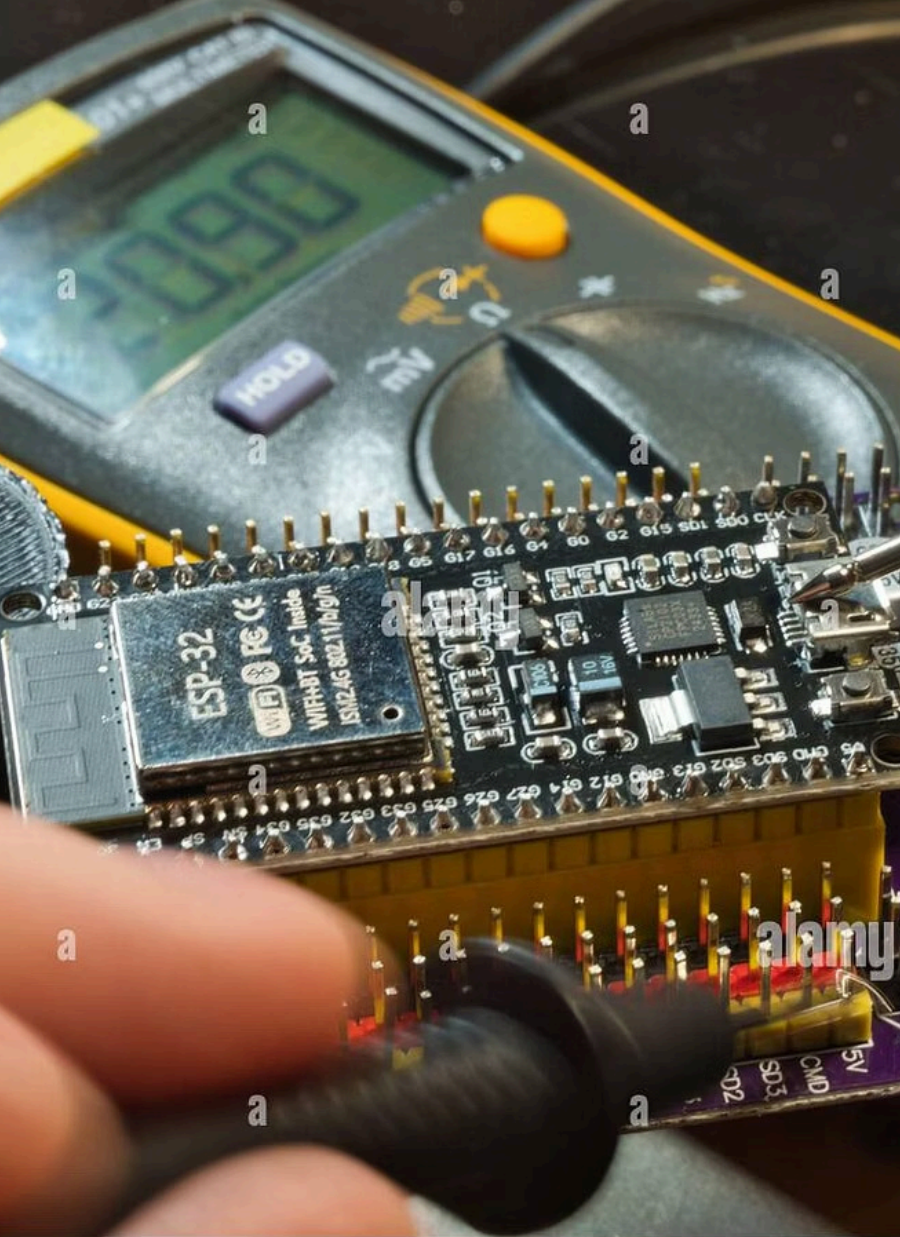




RTOS Middle: Manajemen Task & Interrupt

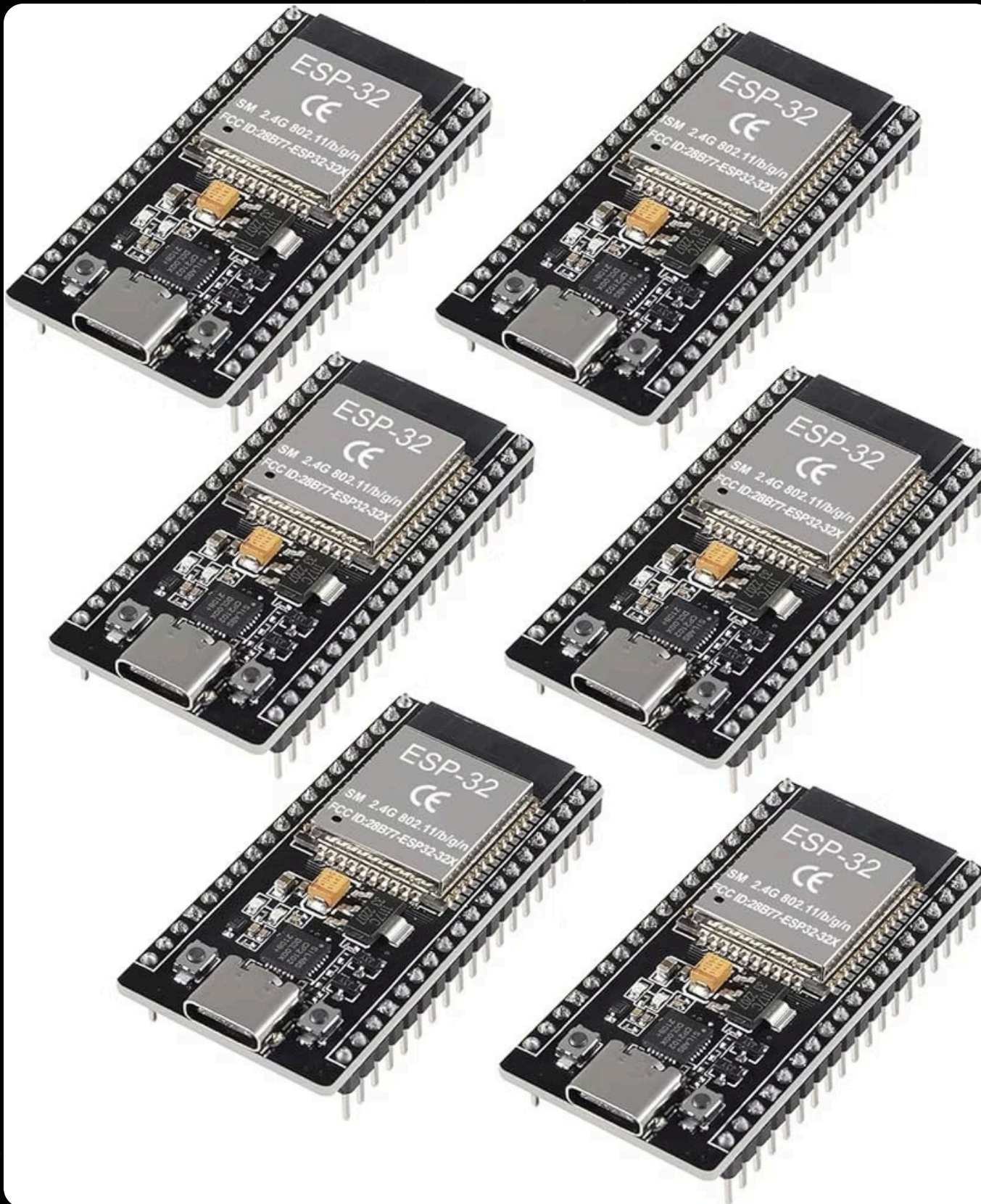
Memasuki tingkat menengah dalam implementasi **Real-Time Operating System (RTOS)** menggunakan mikrokontroler ESP32

FAHREZA ILHAM TANGGUH ADITYA

40040324620007

PRAKTIKUM A

Mengapa ESP32 Ideal untuk RTOS?



ESP32 sangat ideal untuk implementasi RTOS tingkat menengah karena dua alasan utama:

Arsitektur Dual-Core

Dua inti prosesor memungkinkan pembagian beban kerja secara paralel, meningkatkan responsivitas sistem secara signifikan.

Dukungan FreeRTOS Bawaan

FreeRTOS sudah terintegrasi langsung dalam framework ESP32, sehingga tidak perlu instalasi tambahan.

- ❶ Inti dari RTOS pada tingkat ini adalah bagaimana kita menjamin bahwa tugas yang paling kritis dan sensitif terhadap waktu selalu mendapatkan jatah prosesor tepat saat dibutuhkan, tanpa terblokir oleh tugas lain yang lambat.

Dua Percobaan Utama

Materi ini mencakup dua percobaan yang membangun pemahaman mendalam tentang manajemen RTOS pada ESP32.

Percobaan 1

Manajemen Task Priority

Preemptive Scheduling - membagi tugas ke dalam hierarki prioritas agar komponen kritis tidak terhambat oleh tugas yang lambat.

Percobaan 2

Sinkronisasi Interrupt dengan RTOS

Deferred Interrupt Processing - memisahkan eksekusi ISR yang cepat dari tugas berat menggunakan mekanisme Semaphore.



Set project goals for
prioritization

Align leadership to
define "value"

Quantify portfolio
level constraints

Acceleration of key
projects

Percobaan 1: Manajemen Task Priority

Konsep Utama: **Preemptive Scheduling**

Dalam sistem otomasi yang solid, antarmuka pengguna (HMI) atau pembacaan sensor **tidak boleh terhenti (hang)** hanya karena sistem sedang sibuk menulis data ke memori penyimpanan. Proses penulisan modul SD Card via antarmuka SPI dikenal sangat lambat dan bersifat *blocking*. Oleh karena itu, kita membaginya ke dalam tiga hierarki prioritas.

Alokasi Prioritas Task

Tiga komponen sistem dibagi ke dalam hierarki prioritas yang berbeda untuk memastikan responsivitas optimal.

1

● Prioritas Tinggi - LCD I2C

Layar adalah antarmuka visual utama. Pembaruan data di layar harus terasa instan dan tidak boleh terlihat patah-patah (lag) di mata pengguna.

2

● Prioritas Sedang - Sensor DHT

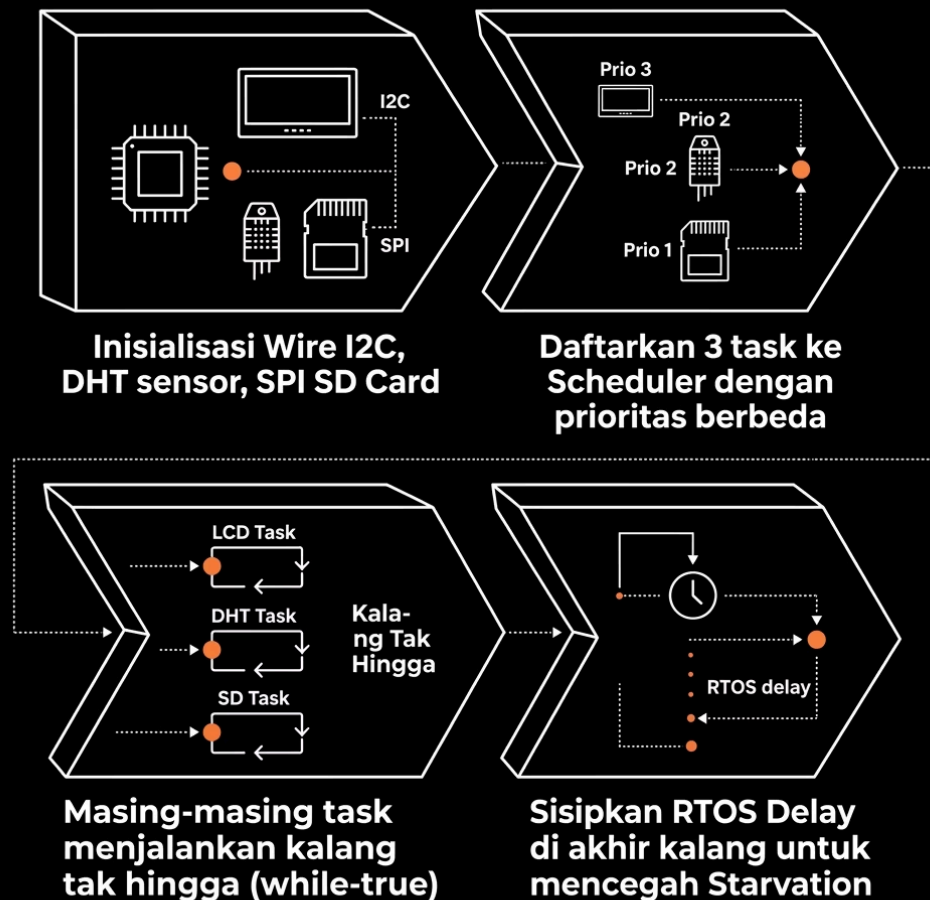
Membaca suhu dan kelembapan dari DHT memerlukan timing yang spesifik dan memakan waktu beberapa milidetik. Tugas ini penting untuk menyediakan data terbaru, namun masih bisa ditunda sepersekian detik oleh pembaruan layar.

3

● Prioritas Rendah - SD Card Reader

Mencatat data (data logging) adalah proses administratif. Jika penulisan ke kartu SD tertunda beberapa milidetik karena sistem sedang sibuk memperbarui layar LCD, hal tersebut sama sekali tidak akan merusak integritas sistem secara keseluruhan.

Alur Program: Inisialisasi & Task



Struktur Program

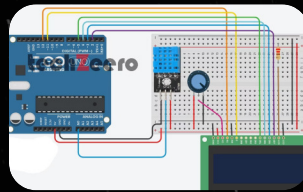
Pada blok inisialisasi awal (fungsi setup), Anda perlu memulai komunikasi untuk setiap komponen, seperti inisialisasi Wire untuk jalur I2C layar LCD, memulai pembacaan sensor DHT, dan mengaktifkan jalur SPI untuk modul kartu SD.

Setelah perangkat keras siap, langkah terpenting adalah memanggil fungsi pembuat task bawaan FreeRTOS secara berurutan. Anda akan mendaftarkan tiga fungsi terpisah kepada scheduler.



Aturan Krusial: Anda harus menyisipkan instruksi penunda (seperti fungsi `delay` milik RTOS) di akhir setiap kalang tak hingga tersebut. Penundaan ini berfungsi untuk menyerahkan kembali kendali CPU dari task berprioritas tinggi kepada task berprioritas rendah agar tidak terjadi *starvation* — kondisi di mana task prioritas rendah tidak pernah dieksekusi.

Logika Internal Setiap Task

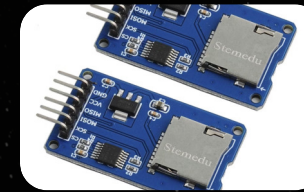


Task LCD (Prioritas 3)

Mengambil data suhu yang telah disimpan oleh task DHT, lalu mencetaknya ke layar I2C secara terus-menerus dalam kalang tak hingga.

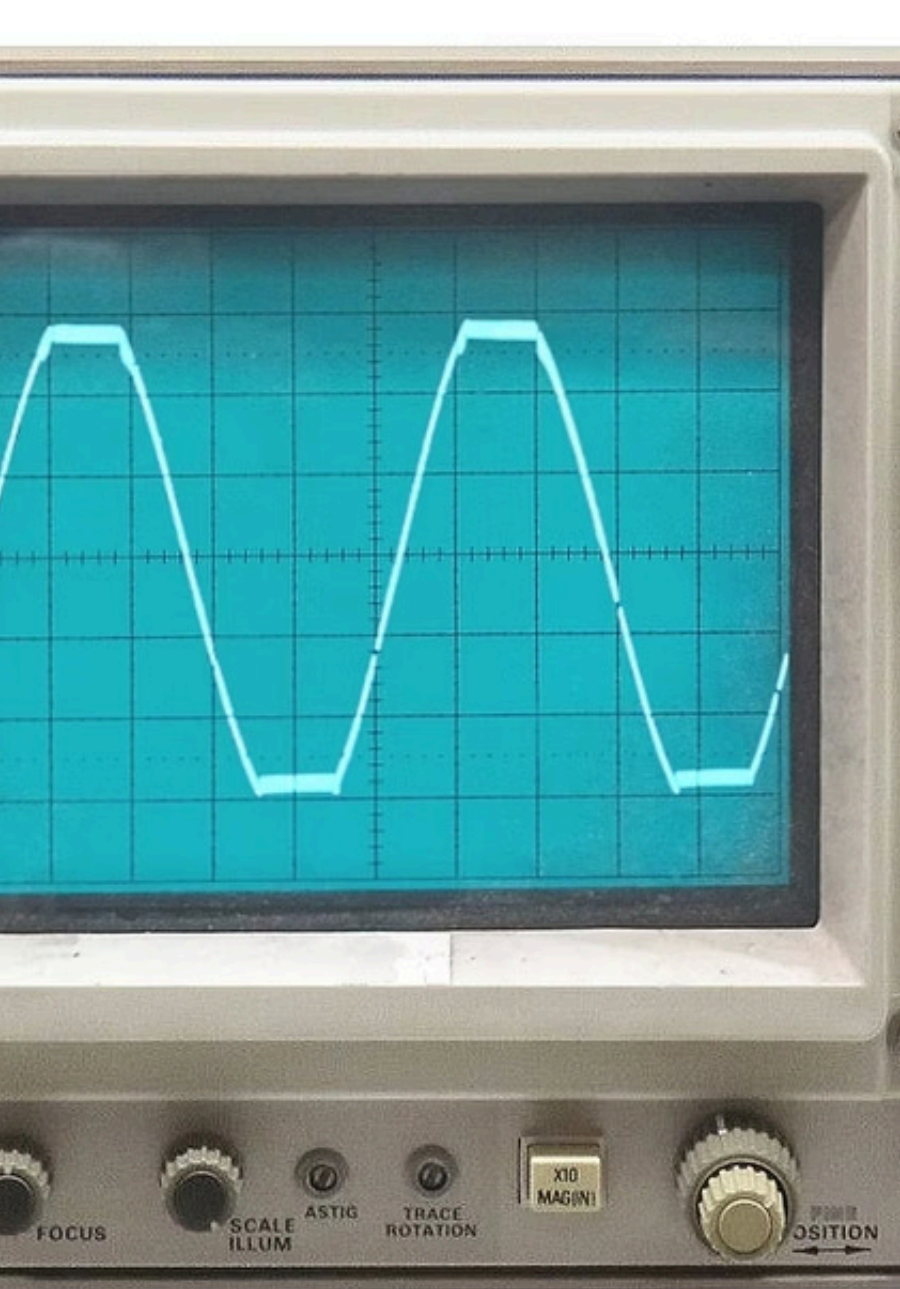
Task DHT (Prioritas 2)

Membaca data suhu dan kelembapan dari sensor DHT, lalu menyimpannya ke variabel global yang dapat diakses task lain.



Task SD Card (Prioritas 1)

Membuka file teks, menulis data log, lalu menutup file tersebut. Proses ini bersifat *blocking* dan memakan waktu paling lama.



Percobaan 2: Sinkronisasi Interrupt dengan RTOS

Konsep Utama: **Deferred Interrupt Processing**

Sering kali, pemrogram pemula mengeksekusi banyak perintah berat langsung di dalam fungsi **Interrupt Service Routine (ISR)**, seperti menyalakan lampu, membaca sensor, hingga mencetak teks. Dalam RTOS, fungsi ISR harus dieksekusi **secepat kilat** (dalam ukuran mikrodetik). Pekerjaan beratnya harus dilempar atau "ditangguhkan" (*deferred*) ke sebuah task biasa yang menunggu sinyal dari interrupt tersebut.

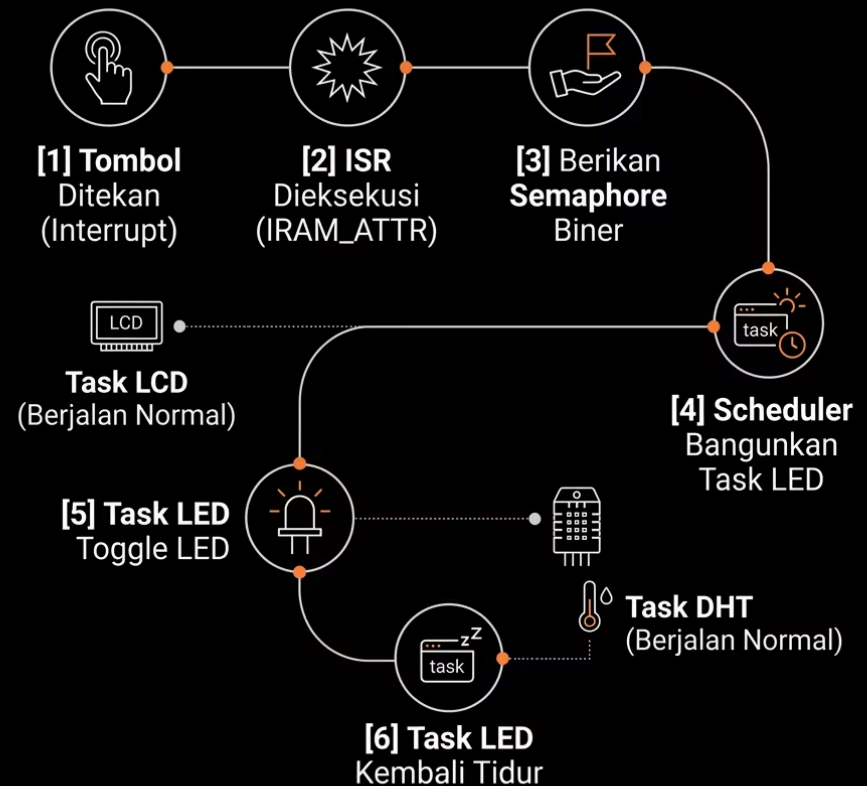
Alur Kerja: ISR + Semaphore

Mekanisme Kerja

Sensor DHT dan layar LCD I2C berjalan normal secara terus-menerus di latar belakang sebagai task reguler. Sistem menunggu masukan tak terduga dari **tactile push button**. Saat tombol ditekan, eksekusi hardware akan membajak prosesor sejenak (interrupt), lalu sistem akan menyalakan atau mematikan LED tanpa sedikit pun mengganggu irama pembacaan sensor DHT dan pembaruan LCD.

- Konfigurasi pin tombol sebagai **input pull-up** dan pin LED sebagai **output**
- Tambahkan makro `IRAM_ATTR` pada fungsi ISR agar disimpan di RAM internal ESP32
- Buat **Semaphore biner** sebagai "tongkat estafet" antara interrupt dan task
- Task LED **tertidur pulas** (tidak memakan CPU) hingga Semaphore diberikan
- ISR hanya **satu baris perintah**: memberikan Semaphore, lalu segera keluar

Alur sinkronisasi ISR dengan RTOS.



Kesimpulan: Dua Pilar RTOS

Pergeseran paradigma dari pemrograman berurutan (sekuensial) menjadi sistem manajemen sumber daya prosesor yang cerdas dan efisien menggunakan RTOS.



Preemptive Scheduling (Percobaan 1)

Dalam sistem yang kompleks, tugas tidak boleh dijalankan asal-asalan. Komponen yang membutuhkan respons visual instan (seperti LCD) mutlak diberi **prioritas tinggi**, sedangkan tugas administratif yang memakan waktu dan bersifat *blocking* (seperti SD Card) harus diberi **prioritas rendah**. Konsep ini, dipadukan dengan jeda waktu (RTOS delay) yang tepat, memastikan tidak ada tugas yang memonopoli CPU sehingga fenomena *starvation* (tugas prioritas rendah tidak kebagian waktu eksekusi) dapat dihindari.



Deferred Interrupt Processing (Percobaan 2)

Eksekusi *Interrupt* (ISR) harus dilakukan secepat kilat agar sistem tidak *hang*. Alih-alih memasukkan perintah berat ke dalam ISR, kita menggunakan mekanisme **Semaphore** sebagai "tongkat estafet". ISR hanya bertugas melempar sinyal, dan eksekusi lanjutan dikerjakan oleh task terpisah di latar belakang. Hal ini memastikan pembacaan sensor dan pembaruan layar tetap berjalan mulus tanpa terganggu oleh interupsi tombol.